

Courier Pickup Scheduler & Tracking

Overview

Build a small backend service that manages courier pickup requests and tracks packages. The system stores depots, drivers, packages and pickup requests locally (SQLAlchemy). It integrates with an external courier API (via [requests](#)) to push pickup requests and fetch tracking updates. Expose basic HTTP endpoints with **FastAPI**. Structure the logic using OOP and include proper logging.

Before coding

Draw the database schema on paper (tables + relationships) and show it for review.

Required Models (SQLAlchemy ORM)

- **Depot**
 - id, name, address, is_active, created_at
- **Driver**
 - id, depot_id (FK → Depot.id), name, phone, is_active
- **Package**
 - id, depot_id (FK), tracking_id (nullable), recipient_name, recipient_address, status (created/in-transit/delivered), created_at, updated_at
- **PickupRequest**
 - id, package_id (FK → Package.id), requested_at, scheduled_at, driver_id (FK → Driver.id, nullable), status (pending/scheduled/completed/cancelled)
- **TrackingEvent** (optional but recommended)
 - id, package_id (FK), event_time, location, description

Relationships:

- Depot 1→* Driver
 - Depot 1→* Package
 - Package 1→* TrackingEvent
 - Package 1→1 PickupRequest (or 1→* if multiple requests are allowed).
-

Core Functional Requirements

1. **CRUD services** (in [services/](#)) for Depot, Driver, Package, PickupRequest.
 - Functions accept a SQLAlchemy [Session](#).
 - Implement **create**, **get_by_id**, list (*filterable by depot/status*), update (*partial*), delete.
2. **OOP Sync/Integration class**
 - [CourierClient](#) (or [CourierSyncService](#)) that:
 - Calls external API endpoints with [requests](#) (simulate with [httpbin.org](#) or a dummy endpoint).

- Methods: `create_remote_pickup(package, scheduled_at)`,
`fetch_tracking(tracking_id)`, `push_status_update(package)`.

3. Logging

- Use Python `logging`.
- Save logs to `logs/app.log` and print to console.
- Use INFO/WARNING/ERROR levels appropriately (API failures, retries, invalid data).

4. Transaction example (mandatory)

- Implement a transactional operation: **Create package + create pickup request + assign to driver** inside one transaction so partial state is not committed if anything fails.

5. FastAPI endpoints (minimal)

- POST `/depots` create depot
- POST `/depots/{depot_id}/packages` create package + pickup request (optionally scheduled)
- GET `/packages/{id}` get package + tracking events
- POST `/packages/{id}/push` trigger push to remote courier (calls `CourierClient`)
- POST `/sync/tracking` fetch latest tracking for all packages with a remote `tracking_id`

Deliverables (GitHub repo — mandatory)

- `models/` — all SQLAlchemy models
- `services/` — CRUD + transactional services
- `integrations/` — `CourierClient` using `requests` (clear, simple methods)
- `main.py` — FastAPI app with the required endpoints
- `demo.py` — script demonstrating:
 - create depot, driver
 - create package + pickup (transactional)
 - push to remote endpoint
 - fetch tracking update
- `README.md` — setup, run, example requests (curl/python snippets)
- `logs/` directory (gitignored except README note)
- Proper `.gitignore` included

Example Project Structure (Only for reference)

```
courier_scheduler/  
├── README.md  
├── demo.py  
├── main.py  
├── logging_config.py  
├── .gitignore  
├── models/  
│   ├── base.py  
│   ├── depot.py  
│   ├── driver.py  
│   └── package.py
```

```
| ├── pickup_request.py
|   └── tracking_event.py
├── services/
|   ├── depot_service.py
|   ├── driver_service.py
|   ├── package_service.py
|   └── pickup_service.py
├── integrations/
|   └── courier_client.py
└── logs/
    └── app.log
```

Terms & Conditions

- Use a proper **GitHub repository**; commit messages must be clear and descriptive.
- Include a proper `.gitignore`.
- **No AI-generated code** permitted. Use Google and official docs if needed. If in doubt, ping mentors.
- Must be able to explain all code during review.